A Project Stage-I Report on

FAKE NEWS DETECTION USING GNNs

Submitted in Partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

AKULA SAI VIJAYA	**23BD1A1203**
BODHANKAR ADITYA	**23BD1A1212**
KARNATI ABHIGNYA	
SHANTHI DURGA	**23BD1A1228**
PRANATHI BAREDDY	**23BD1A1253**

Under the guidance of

Ms. T N SRI RANJANI
Assistant Professor, Department of IT



DEPARTMENT OF INFORMATION TECHNOLOGY

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29
2026-27

.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that this is a bonafide record of the project report titled **"FAKE NEWS DETECTION USING GNNs"** which is being presented as the **Project Stage-I** report in **IV year I Semester (KR23)** by

1. AKULA SAI VIJAYA	**23BD1A1203**
2. BODHANKAR ADITYA	**23BD1A1212**
3. KARNATI ABHIGNYA SHANTHI DURGA	**23BD1A1228**
4. PRANATHI BAREDDY	**23BD1A1253**

In partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to the Jawaharlal Nehru Technological University Hyderabad, Hyderabad

Faculty Supervisor
(Ms T N SRI RANJANI)

Head of Department
(Mr. Para Upendar)

Vision & Mission of KMIT

Vision of the Institution

- To be the fountain head in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission of the Institution

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

Vision & Mission of CSE

Vision of the Department

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department

- To provide faculty with state of the art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefit the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities

PROGRAM OUTCOMES (POs)

PO1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem Analysis: Identify formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences

PO3. Design/Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct Investigations of Complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern Tool Usage: Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The Engineer and Society: Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7. Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Information Technology solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep learning, IOT, Data Science, Full stack development, Social Networks, Cyber Security, Mobile Apps, CRM, ERP, Big Data, etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: To imbibe analytical and professional skills for successful careers and create enthusiasts to pursue advance education supplementing their career growth.

PEO2: Graduates will solve real time problems design, develop and implement innovative ideas by applying their computer engineering principles.

PEO3: Graduates will develop necessary skillset for industry by imparting state of art technology in various areas of computer science engineering.

PEO4: Graduates will engage in lifelong learning and be able to work collaboratively exhibiting high level of professionalism.

DECLARATION

We hereby declare that the results embodied in the dissertation entitled **"FAKE NEWS DETECTION USING GNNs"** has been carried out by us together during the academic year 2026-27 as a partial fulfillment of the award of the B.Tech degree in Computer Science and Engineering from Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to JNTUH. We have not submitted this report to any other university or organization for the award of any other degree.

Student Name	Rollno.
AKULA SAI VIJAYA	23BD1A1203
BODHANKAR ADITYA	23BD1A1212
KARNATI ABHIGNYA SHANTHI DURGA	23BD1A1228
PRANATHI BAREDDY	23BD1A1253

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte**, Founder & Director, **Mr. S. Nitin,** Director for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Director Academic for providing an excellent environment in the college.

We are also thankful to **Mr. Para Upendar**, Head of the Department for providing us with time to make this project a success within the given schedule.

We are also thankful to our Faculty Supervisor **Ms. SRI RANJANI**, for her/his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire CSE Department faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

Student Name	Rollno.
AKULA SAI VIJAYA	23BD1A1203
BODHANKAR ADITYA	23BD1A1212
KARNATI ABHIGNYA SHANTHI DURGA	23BD1A1228
PRANATHI BAREDDY	23BD1A1253

ABSTRACT

Fake news spreads faster than manual fact-checking can follow, and many automated detectors still judge a story from its text alone, give no evidence for their verdicts, or treat retrieved documents as unrelated pieces of text. This project presents GraphVerify, an explainable multimodal fake news detection system that combines a knowledge graph, a Graph Attention Network (GAT), retrieval-augmented generation (RAG) and a large language model (LLM). It extends the work of Harris et al. (Journal of Intelligent Information Systems, 2026), whose M&M-RAG framework retrieved evidence from credible sources and passed it to Qwen2-VL, reaching 94.6% accuracy and 94.2% F1 but processing each retrieved snippet independently. GraphVerify accepts a news article with an optional image. BERT embeds the text and a vision model extracts image features. Entities are then extracted to build a knowledge graph linking the article, its entities, events and evidence, and a GAT learns which relationships matter most and gives an initial Fake/Real prediction. In parallel, the extracted entities query a FAISS vector database built from trusted news, government and fact-check sources. An LLM (Llama/Mistral) then applies an evidence-first policy. It confirms the GAT prediction when strong credible evidence supports it, revises it when strong evidence contradicts it, and returns "inconclusive" when evidence is weak or conflicting. The output is a Fake/Real label, a confidence score, the supporting sources, and a verification report that shows the reasoning path from claim to entity to evidence to verdict. By modelling relationships among claims and evidence rather than reading them in isolation, the system aims to improve accuracy, transparency and real-time verification over text-only and RAG-only approaches.

LIST OF FIGURES

S.No	Name of Screenshot	Page No.
1.	Fig 1.1 Architecture diagram	5
2.	Fig 4.1 Class diagram	19
3.	Fig 4.2 Sequence diagram	20
4.	Fig 4.3 State chart diagram	21
5.	Fig 4.4 Deployment diagram	21

CONTENTS

DESCRIPTION	PAGE
CHAPTER - 1	**1**
1. INTRODUCTION	**2-5**
1.1. Purpose of the project	2
1.2. Problem with Existing System	3
1.3. Proposed System	3
1.4. Scope of the Project	4
1.5. Architecture Diagram	5
CHAPTER – 2	**6**
2. LITERATURE SURVEY	**7-9**
CHAPTER - 3	**10**
3. SOFTWARE REQUIREMENT SPECIFICATION	**11-13**
3.1. Introduction to SRS	11
3.2. Role of SRS	11
3.3. Requirements Specification Document	11
3.4. Functional Requirements	13
3.5. Non-Functional Requirements	13-14
3.6. Performance Requirements	15
3.7. Software Requirements	15
3.8. Hardware Requirements	16
CHAPTER-4	**17**
4. SYSTEM DESIGN	**18-21**
4.1. Introduction to UML	18
4.2. UML Diagram	18
4.2.1. Class Diagram	18

4.2.2. Sequence Diagram 19
4.2.3. State Chart Diagram 21
4.2.4. Deployment Diagram 21
4.3. TECHNOLOGIES USED 22
REFERENCES **23**

CHAPTER-1

1. INTRODUCTION

Social media has made it possible for news to reach millions of people within seconds, but it has also made it easy for fake news to spread just as fast. Misinformation can mislead the public, influence opinions and elections, and cause social and political harm. With thousands of articles and posts shared every day, manual fact-checking cannot keep pace, so automated detection systems are essential. Early detectors relied on text classification with models such as BERT, but these judge only how an article reads and not whether its claims are true. Graph-based methods such as Graph Attention Networks improved matters by learning relationships between entities, yet they offer no external evidence and little explanation. Retrieval-Augmented Generation (RAG) systems address this by retrieving trusted evidence before making a prediction. The base paper by Harris et al. (2026) combines RAG with a vision-language model for multilingual, multimodal detection and reaches 94.6% accuracy. However, it processes each retrieved document independently and does not model how claims, entities and evidence relate to one another. This project proposes a system that combines a knowledge graph, a Graph Attention Network, RAG and a large language model to close that gap. It builds a graph of the claim and its evidence, learns contextual relationships, and lets the LLM verify the result against retrieved sources.

1.1. Purpose of the Project

The purpose of this project is to build an automated fake news detection system that is accurate, evidence-based and explainable The specific aims are:

- **Capture relationships:** build a knowledge graph linking the news article, its entities, events and retrieved evidence, instead of treating each document in isolation.
- **Improve accuracy:** use a Graph Attention Network to learn which relationships matter most for judging credibility.
- **Ground predictions in evidence:** use RAG to retrieve real-time evidence from trusted news, government and fact-check sources, reducing reliance on the LLM's internal knowledge and the risk of hallucination.
- **Explain every verdict:** produce a Fake/Real label with a confidence score, supporting sources and a reasoning path from claim to entity to evidence to verdict.
- **Support users and platforms:** help social media platforms, news organisations and ordinary readers identify misleading information early and reduce its spread.

1.2. Problem with Existing Systems

Existing approaches to multimodal RAG framework of Harris et al have the following problems:

- **Evidence is treated in isolation:** each retrieved document is processed independently, so 2 the system cannot see how the claim, entities, events and evidence connect.
- **No structured knowledge:** it does not use a knowledge graph, so entities and events are never represented in a form that supports graph-based reasoning.
- **Weak explanations:** the LLM writes a text justification, but it does not give an explicit evidence path showing how the verdict was reached, which makes the explanation harder to verify.
- **Nothing is reused:** because there is no reusable knowledge structure, fresh retrieval is needed for every query, which adds cost and delay
- **Dependence on retrieval quality:** performance depends heavily on the documents retrieved. The base paper itself reports errors from off-topic or weak evidence, especially for low-resource languages, and from evidence that is topically related but factually insufficient.

1.3. Proposed System

GraphVerify keeps the evidence-retrieval and LLM-verification core of the base paper and adds a knowledge graph and a Graph Attention Network (GAT) that model how a claim relates to its entities and evidence. A request is processed in the following steps:

- **Input:** the user submits a news article as text, with an optional image.
- **Preprocessing:** the text is cleaned, tokenised and converted into semantic embeddings with BERT, and the image is analysed by a vision model. Together these capture the contextual meaning of the news content.
- **Graph construction:** entities are extracted from the article and used to build a knowledge graph. Nodes represent the article and its related entities, and edges represent the relationships between them.
- **Graph analysis:** a Graph Attention Network learns how important each relationship is and makes an initial Fake/Real prediction about the article's credibility.
- **Evidence retrieval:** the extracted entities are sent to the RAG retriever, which searches a FAISS vector database built from trusted news articles, government sources and fact-check repositories, and returns the evidence most relevant to the claim.
- **LLM verification:** an LLM (Llama/Mistral) receives both the GAT prediction and the retrieved evidence and applies an evidence-first policy..

- **Output:** the system returns the Fake/Real label (or inconclusive) with a confidence score and the supporting evidence. It also gives a verification report showing the reasoning path from claim to entity to evidence to verdict.

Area	Base paper (Harris et al., 2026)	GraphVerify
Knowledge base	None; each retrieved snippet is treated separately	Knowledge graph linking the article, entities, events and evidence
Reasoning model	Frozen Qwen2-VL, single-pass, zero-shot	GAT learns relationships, then an LLM (Llama/Mistral) verifies
Input	Text claim and one image	Text and optional image (BERT and a vision model)
Evidence retrieval	Claim text sent to live search engines (Google, Bing, Baidu); LaBSE similarity ranking	Extracted entities query a FAISS vector database of trusted sources
Explanation	Free-text justification written by the LLM	Verification report with a reasoning path: claim → entity → evidence → verdict

Table 1.1 Enhancements of GraphVerify over the base paper

1.4. Scope of the Project

In scope. GraphVerify is a web application for explainable fake news detection. It accepts a news article as text with an optional image and returns a Fake, Real or Inconclusive verdict with a confidence score. Each verdict comes with the supporting evidence and a verification report showing the reasoning path from claim to entity to evidence to verdict. The system includes an input and preprocessing module (BERT text embeddings and a vision model for images), an entity extraction and knowledge graph construction module, a Graph Attention Network that predicts credibility, a RAG retriever over a FAISS vector database built from trusted news, government and fact-check sources, and an LLM (Llama/Mistral) that verifies the prediction using an evidence-first policy.

Out of scope. The system does not decide what is true on its own authority, and it does not replace professional fact-checkers or editorial judgement. It does not detect fake videos, audio or deepfakes, and it does not analyse how news spreads across social networks. Its knowledge is limited to the documents in the vector database, so claims about very recent events may return "Inconclusive" until new sources are added.

Stage II. The next stage will evaluate the system on a held-out test set, reduce inference time, expand the trusted-source database, and deploy the same pipeline as a hosted web service.

1.5. Architecture Diagram

The GraphVerify architecture is organised into five layers: input, preprocessing, graph analysis, knowledge retrieval, and verification and output. The user enters a claim or news article, with an optional image, in the React frontend. The request travels over HTTPS through Nginx to the FastAPI backend, which validates it and starts the pipeline. In the preprocessing layer, the text is cleaned and tokenised and converted into semantic embeddings with BERT, while a vision model extracts features from the image. In the graph analysis layer, entities are extracted from the article and used to build a knowledge graph, whose nodes are the article and its entities and whose edges are their relationships. A Graph Attention Network (GAT) learns the importance of these relationships and makes an initial Fake or Real prediction. In the knowledge retrieval layer, the system retrieves relevant articles, searches the web and the knowledge base, and ranks the evidence from trusted sources. In the verification and output layer, the LLM receives the GAT prediction together with the ranked evidence and applies an evidence-first policy.

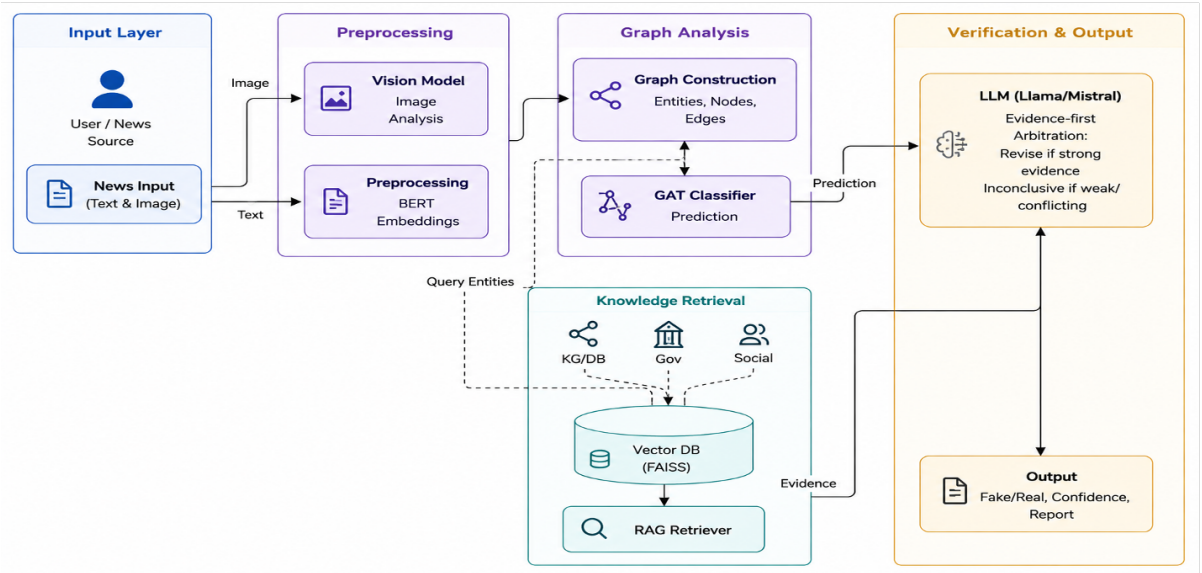


Fig 1.1 Architecture diagram

CHAPTER-2

2. LITERATURE SURVEY

This chapter reviews the work on which GraphVerify is built: the base paper, retrieval-augmented generation, text and multimodal fake news detectors, graph-based detection, LLM-based evidence verification, and vector search for retrieval.

1. S. Harris, V. T. Ta, M. Trovati, G. Nakhla, F. Latif and I. Korkontzelos, "Multimodal Misinformation Detection Across Diverse Languages Using RAG and LLMs", Journal of Intelligent Information Systems, Springer, 2026 (base paper)..

The authors proposed M&M-RAG, which embeds a news claim with LaBSE, retrieves evidence from live search engines (Google, Bing, Baidu), filters it with a domain whitelist and keeps the top five snippets. These are passed with the claim and image to a frozen Qwen2-VL model, which returns a Fake/Real label with a short justification. The system reached 94.6% accuracy and 94.2% F1 across English, Chinese and Urdu datasets without any fine-tuning. Gap: retrieved snippets are processed independently, no knowledge graph or graph reasoning is used, and the explanation is free text without an explicit evidence path, which GraphVerify addresses.

2. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS, 2020.

Introduced RAG, which combines a neural retriever with a generator so that answers are conditioned on retrieved documents. RAG improves factual accuracy and lets the knowledge be updated without retraining. GraphVerify follows this design and additionally structures the retrieved evidence as a graph.

3. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL, 2019.

BERT learns contextual text representations from large unlabelled corpora and is widely used for text classification, including fake news detection. Gap: a text-only classifier judges how an article reads and cannot check its claims against external evidence. GraphVerify uses BERT only to embed the text, not to decide the verdict.

4. P. Veličković et al., "Graph Attention Networks", ICLR, 2018.

Introduced the Graph Attention Network (GAT), which uses attention to give each node's neighbours different weights, so the model learns which relationships matter most. GraphVerify uses a GAT over the claim–entity graph to predict credibility.

5. Y. Wang et al., "EANN: Event Adversarial Neural Networks for Multi-Modal Fake News Detection", ACM SIGKDD, 2018. Learned event-invariant features from text and images through adversarial training, reducing overfitting to specific events. Gap: it uses no external evidence and no relationships between entities, so its predictions cannot be verified.

6. S. Singhal et al., "SpotFake: A Multi-modal Framework for Fake News Detection", IEEE BigMM, 2019.

Fused BERT text features with VGG-19 image features for classification on Twitter and Weibo data. Gap: early feature fusion with no evidence retrieval or explanation.

7. J. Jing et al., "Multimodal Fake News Detection via Progressive Fusion Networks", Information Processing & Management, 2023.

Proposed a multi-tier fusion network that captures modality-specific representations of text and images. The base paper notes that it must be trained on each dataset and lacks zero-shot ability, interpretability and external knowledge.

8. S. Nie and Z. Zeng, "Towards Real-World Multimodal Propagation Networks: Multimodal Uncertainty Graph Contrastive Learning for Fake News Detection", Journal of Intelligent Information Systems, 2025 (MUGCL).

Combined text, images and social propagation networks in a contrastive learning framework, building dual propagation graphs and filtering out deceptive signals. It shows the value of graph learning for fake news. Gap: its graphs model how news spreads, not the facts in the claim, and it does not verify claims against retrieved evidence.

9. B. M. Yao et al., "End-to-End Multimodal Fact-Checking and Explanation Generation: A Challenging Dataset and Models", ACM SIGIR, 2023 (MOCHEG).

Combined evidence retrieval, claim verification and explanation generation for multimodal news. According to the base paper, it did not implement a true RAG architecture and struggled with complex evidence selection.

10. S. Tahmasebi, E. Müller-Budack and R. Ewerth, "Multimodal Misinformation Detection Using Large Vision-Language Models", ACM CIKM, 2024.

Used a vision-language model in a zero-shot setting with CLIP-based visual evidence retrieval. It shows that large models can detect misinformation without training, but the evidence is still handled as separate items.

Summary of the Survey

The survey shows that text and multimodal classifiers detect fake news well but cannot check claims against evidence. Graph-based methods learn useful relationships but give no external verification or explanation, and RAG-based systems retrieve evidence but treat it as isolated documents. No single existing system combines graph learning, evidence retrieval and LLM verification with a clear reasoning path. GraphVerify combines these elements in one pipeline.

CHAPTER-3

3. SOFTWARE REQUIREMENT SPECIFICATION

1. Introduction to SRS

This Software Requirements Specification (SRS) describes the requirements of GraphVerify, an explainable multimodal fake news detection system. It defines what the system must do, the inputs it accepts, the outputs it produces and the constraints it works under. It is intended for the project guide, the development team and the evaluators. It serves as the reference for the design, implementation and testing of the system.

2. Role of SRS

The SRS acts as an agreement between the project team, the guide and the users of the system. It is used to:

- give all team members a common understanding of the system to be built;
- form the basis for design, implementation and the division of work;
- define the test cases and the evaluation metrics against which the system is verified;
- control changes, because every new feature can be checked against the agreed requirements;
- record the safety limits of the system, which is particularly important for a healthcare application.

3. Requirements Specification Document

Product perspective. GraphVerify is a client-server web application. A browser-based frontend sends the news text and optional image through a JSON REST API to a Python backend. The backend hosts the preprocessing, knowledge graph construction, GAT, retrieval and LLM verification components, and uses a FAISS vector database as the evidence store. It extends the RAG-and-LLM approach of the base paper (Harris et al., 2026) with graph-based reasoning.

Users. (1) General users, such as readers, journalists and platform moderators, who submit a news article and read the verdict, confidence score, supporting evidence and verification report. (2) Administrators or developers, who build and update the evidence database from trusted news, government and fact-check sources, manage the trusted-source list, train the GAT, and run evaluations.

Operating environment. Windows 10/11, Linux or macOS; Python 3.10 or newer with PyTorch and a graph learning library; a modern browser. A GPU is recommended for GAT training and LLM inference, but not required for the frontend.

Constraints and assumptions. The LLM (Llama/Mistral), the BERT and vision models, and the vector database run locally or on the project server. An internet connection is needed for the one-time model downloads and for refreshing the evidence database. Verdicts depend on the sources in the vector database, so claims outside its coverage may return "Inconclusive". Combining graph construction, GAT processing, retrieval and LLM inference makes each request slower than a single-model classifier. The system supports decision-making only and does not replace human fact-checkers, so every result is shown with a note that it should be verified for high-stakes use.

4. Functional Requirements

ID	Requirement
FR1	The system shall accept a news article as text, with an optional image, through the web interface.
FR2	The system shall clean and tokenise the text and generate semantic embeddings with BERT.
FR3	The system shall extract features from the image with a vision model when an image is provided.
FR4	The system shall extract entities from the article and construct a knowledge graph whose nodes are the article and its entities and whose edges are their relationships.
FR5	The system shall apply a Graph Attention Network to the knowledge graph and produce an initial Fake or Real prediction.
FR6	The system shall retrieve the most relevant evidence for the extracted entities from the FAISS vector database of trusted news, government and fact-check sources.
FR7	The system shall pass the GAT prediction and the retrieved evidence to the LLM (Llama/ Mistral) for verification.
FR8	The system shall apply the evidence-first policy: confirm the GAT prediction when strong credible evidence supports it, revise it when strong evidence contradicts it, and return "Inconclusive" when the evidence is weak or conflicting.
FR9	The system shall return the final verdict (Fake, Real or Inconclusive) with a confidence score and the list of supporting sources.
FR10	The system shall generate a verification report showing the reasoning path from claim to entity to evidence to verdict.
FR11	The system shall store the draft answer, sources, safety result and per-stage timings of every message for auditing.
FR12	The system shall provide scripts to build and update the vector database, train the GAT, and evaluate the system against the RAG-only baseline.

Table 3.1 Functional requirements

5. Non-Functional Requirements

- **Accuracy and reliability of verdicts:** the system must base every Fake or Real verdict on retrieved evidence and must not rely on the LLM's internal knowledge alone. When the evidence is weak, conflicting or missing, it must return "Inconclusive" instead of guessing.
- **Explainability:** each result must show the verdict, a confidence score, the supporting sources and a verification report with the reasoning path from claim to entity to evidence to verdict.
- **Security and privacy:** access to the system should be controlled, and submitted articles and stored results should be protected from unauthorised access. Models and the evidence store run locally or on the project server, so submitted content is not sent to third-party AI

services.

- **Performance:** a request passes through graph construction, GAT processing, retrieval and LLM inference, so it takes longer than a single-model classifier. The response time should be measured and reported, and the evidence retrieval should stay fast as the vector database grows.
- **Explainability:** each answer shows its triage level, confidence and sources.
- **Maintainability:** configuration of the trusted-source list, confidence thresholds and arbitration rules through editable files or environment variables, with the pipeline split into separate modules (preprocessing, graph, GAT, retrieval, LLM) that can be tested and updated independently.
- **Portability:** runs on Windows, Linux and macOS, with a GPU recommended for GAT training and LLM inference.

6. Performance Requirements

Measure	Requirement	Measured result
Verdict accuracy	Higher than the RAG-only baseline (the base paper reports 94.6%)	96.5%
Precision, recall and F1 (Fake and Real)	Recall = 1.0	Precision 1.00 and recall 1.00
Retrieval quality (300 MedQuAD questions)	Recall@5 of at least 0.90	Recall@1 0.84, Recall@5 0.93, MRR 0.89
Triage accuracy (55 labelled cases)	At least 0.85	0.93 (English 0.91, Hindi 0.91, Telugu 1.00)
Typed question, full pipeline on CPU	Mean under 60 s	Mean 31 s, 95th percentile 74 s
Memory	Fits in 16 GB RAM	About 8 GB for all models together

Table 3.2 Performance requirements and measured results

7. Software Requirements

Component	Software
Operating system	Windows 10/11, Ubuntu Linux or macOS
Programming languages	Python 3.12
Text and image features	BERT (text embeddings), ViT
Entity extraction	spaCy
Knowledge graph and GAT	NetworkX, PyTorch, PyTorch Geometric
Vector database and retrieval	FAISS
Language and speech	Llama
Web interface and backend	FastAPI and React
Database	MongoDB Community Server (version 8.0 used)
Testing and tools	pytest, mongomock, Git, VS Code

Table 3.3 Software requirements

8. Hardware Requirements

Component	Minimum	Used in this project
Processor	4-core 64-bit CPU	Intel Core i5-13420H
RAM	16 GB	16 GB
Storage	15 GB free (models, packages, data)	SSD
GPU	Not required	Not used
Peripherals	Microphone and speakers for voice	Laptop microphone and speakers
Network	Internet for setup and gTTS	Broadband

Table 3.4 Hardware requirements

CHAPTER-4

4. SYSTEM DESIGN

4.1. Introduction to UML

The Unified Modeling Language (UML) is a standard visual language for specifying, constructing and documenting the parts of a software system. It was adopted as a standard by the Object Management Group (OMG). UML diagrams fall into two groups: structural diagrams, such as class and deployment diagrams, which show the static parts of a system, and behavioural diagrams, such as use case, sequence and state chart diagrams, which show how the system behaves over time. The following sections describe MedRAG+ with these diagrams.

4.2. UML Diagrams

4.2.1. Class Diagram

A class diagram shows the main classes of a system, their attributes and operations, and the relationships between them. The GraphVerify class diagram has three groups of classes. The core classes are `User`, `Claim` and `VerificationResult`. A `User` (with an id, name, email and role) can log in, submit a claim for verification and view the results. One user can submit many `Claim` objects. Each `Claim` holds the claim text, its creation time and a status. It can be analysed, and its result and explanation can be retrieved. Each claim has exactly one `VerificationResult`, which stores the verdict and the confidence score. It can generate a report and return the signals behind the decision. The four pipeline components do the processing for a claim. `BERTRetriever` encodes the text and retrieves the top-k relevant items. `GraphReasoning` builds the graph, applies the GAT and returns a prediction. `WebEvidence` searches the web, fetches articles and ranks the evidence. `LLMVerifier` verifies the claim, generates the explanation and returns the final verdict. The data model classes store the knowledge. An `Article` (title, content and source) is linked to many `GraphNode` objects (entities with a type and name), and nodes are connected by `Edge` objects that record the source node, the target node and the relation between them, with a weight. The `Database` class saves, fetches, updates and deletes these records.

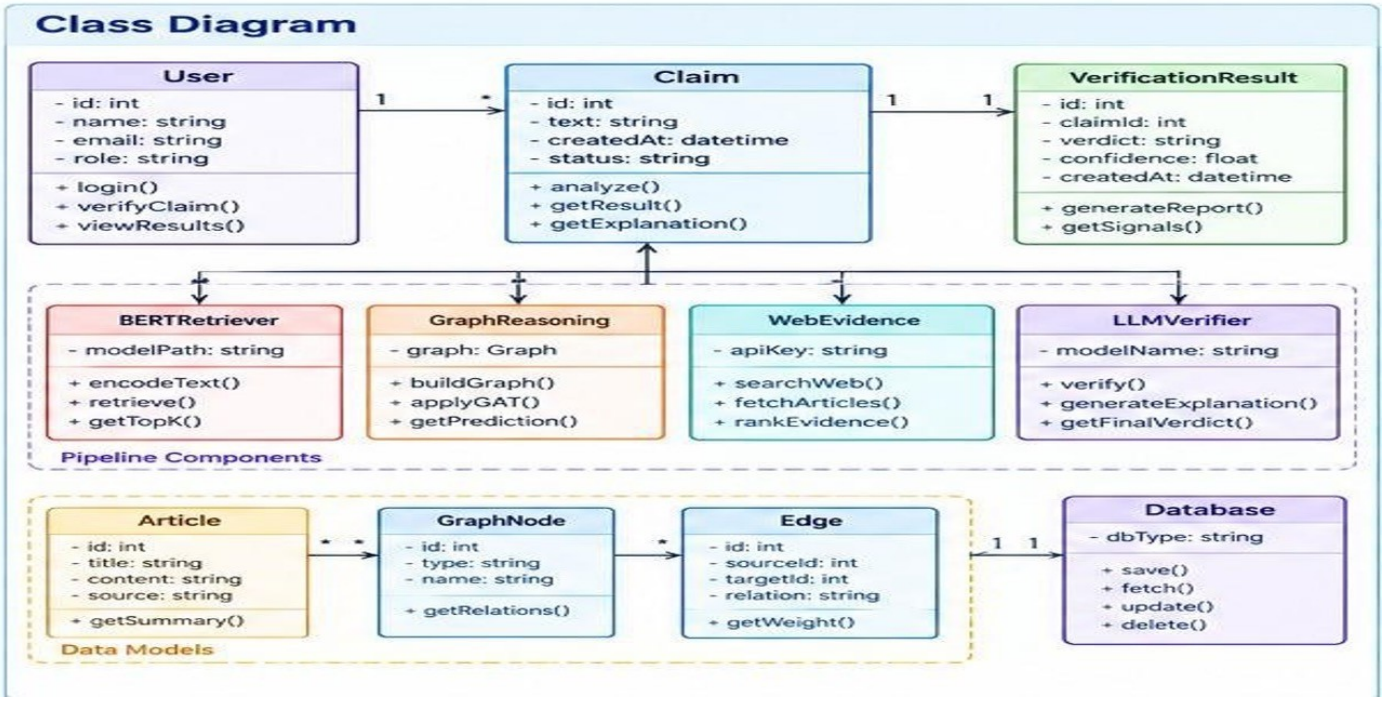


Fig 4.1 Class diagram

4.2.2. Sequence Diagram

A sequence diagram shows how the actors and components of a system exchange messages over time to complete one task. This diagram follows the Verify Claim flow of GraphVerify, which involves five participants: the User, the Frontend (React), the Backend (FastAPI), the Pipeline Services and the Database. The user enters a claim in the web interface and clicks Verify. The frontend sends the claim text to the backend with a `POST /verify` request. The backend validates the request and fetches the data it needs from the database. It then calls the pipeline services in four steps, each of which returns its output before the next one starts. First, the services retrieve relevant articles using BERT embeddings. Second, they build the graph and apply the Graph Attention Network (GAT). Third, they search the web for evidence related to

the claim. Fourth, they use the LLM to generate the verdict. The backend returns the result, containing the verdict and the explanation, to the frontend, which displays it to the user. The user therefore sees only the first and last steps, while the pipeline stages run behind the backend.

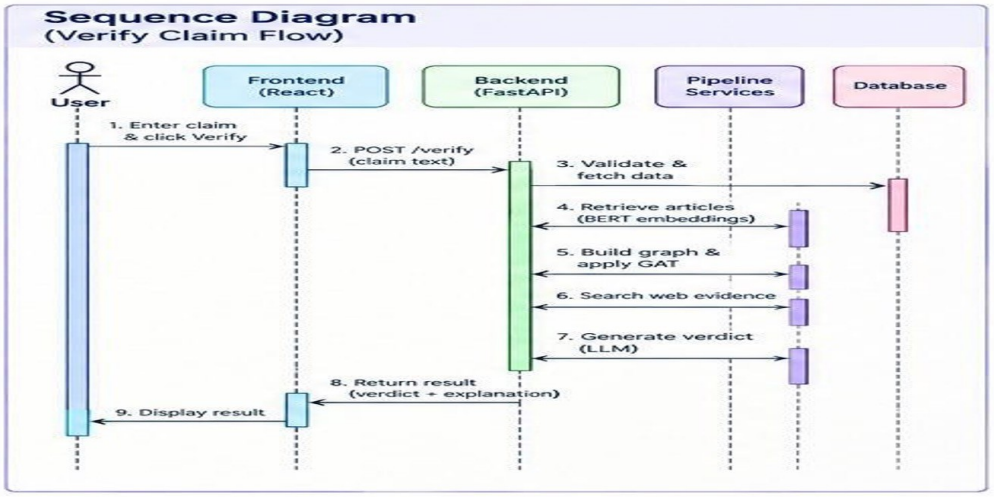
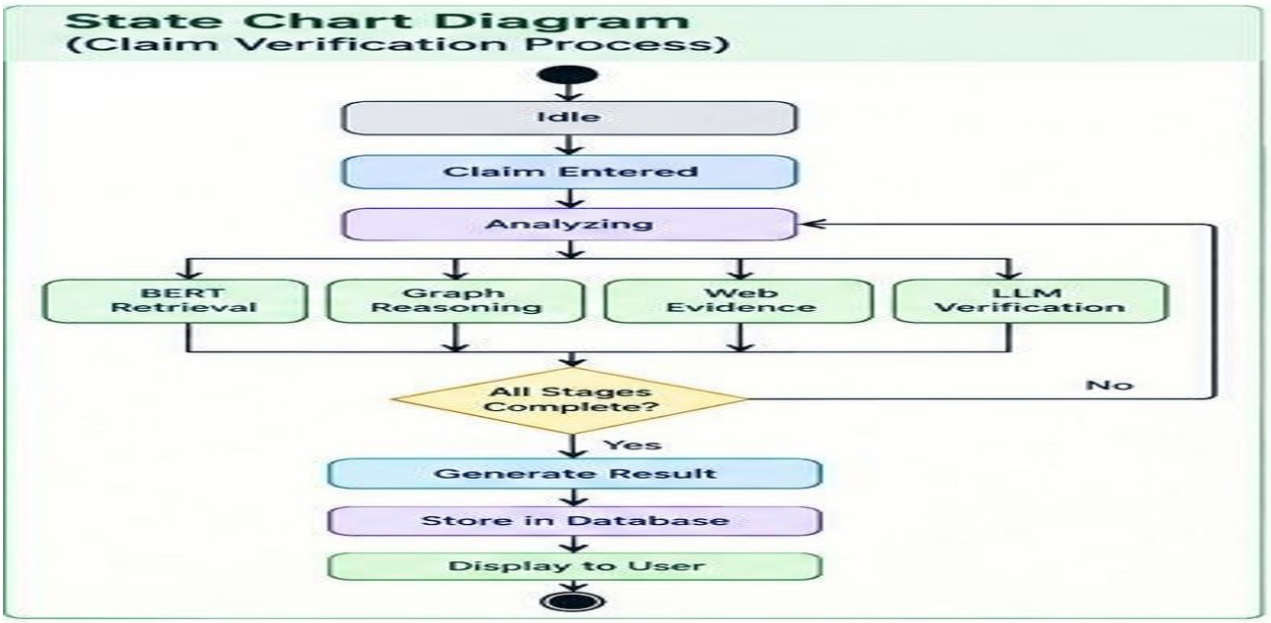


Fig 4.2 Sequence diagram

4.2.3. State Chart Diagram

A state chart diagram shows the states a system passes through and the events that move it from one state to the next. This diagram follows a claim in GraphVerify from entry to display. The system starts in the **Idle** state. When the user enters a claim, it moves to **Claim Entered** and then to **Analyzing**. In the Analyzing state, four processing stages run: **BERT Retrieval** (encoding the claim and retrieving relevant articles), **Graph Reasoning** (building the graph and applying the GAT), **Web Evidence** (searching, fetching and ranking evidence) and **LLM Verification** (verifying the claim and producing the explanation). When these stages finish, the system reaches a decision point, **All Stages Complete?**. If the answer is No, it returns to Analyzing and continues processing. If the answer is Yes, it moves to **Generate Result**, where the verdict, confidence score and explanation are produced. It then moves to **Store in Database**, where the result is saved. Finally it reaches **Display to User**, where the result is shown, and the process ends.

Fig 4.3 State chart diagram



4.2.4. Deployment Diagram

A deployment diagram shows the physical nodes on which the system runs and how they communicate. The GraphVerify deployment has three parts. The **Client Device** is the user's web browser (React frontend) on a desktop or a mobile device with a responsive interface. It talks to the server over **HTTPS**. The **Cloud / Server** node hosts the system. A **Web Server (Nginx)** receives incoming requests and passes them to the **Application Server (FastAPI)**, which runs the verification pipeline. The application server uses two stores. **Model Storage** holds the pre-trained models, and a **Database (PostgreSQL)** holds the application data, such as claims and results. The **External Services** node is reached through **API calls** from the server. It includes a **Web Search API** for news and articles, a **Knowledge Base** of articles and documents, and an **LLM Service** for verification and explanation. A user's request therefore travels from the browser through Nginx to FastAPI, which loads the models, reads and writes the database, and calls the external services before the result goes back the same way.

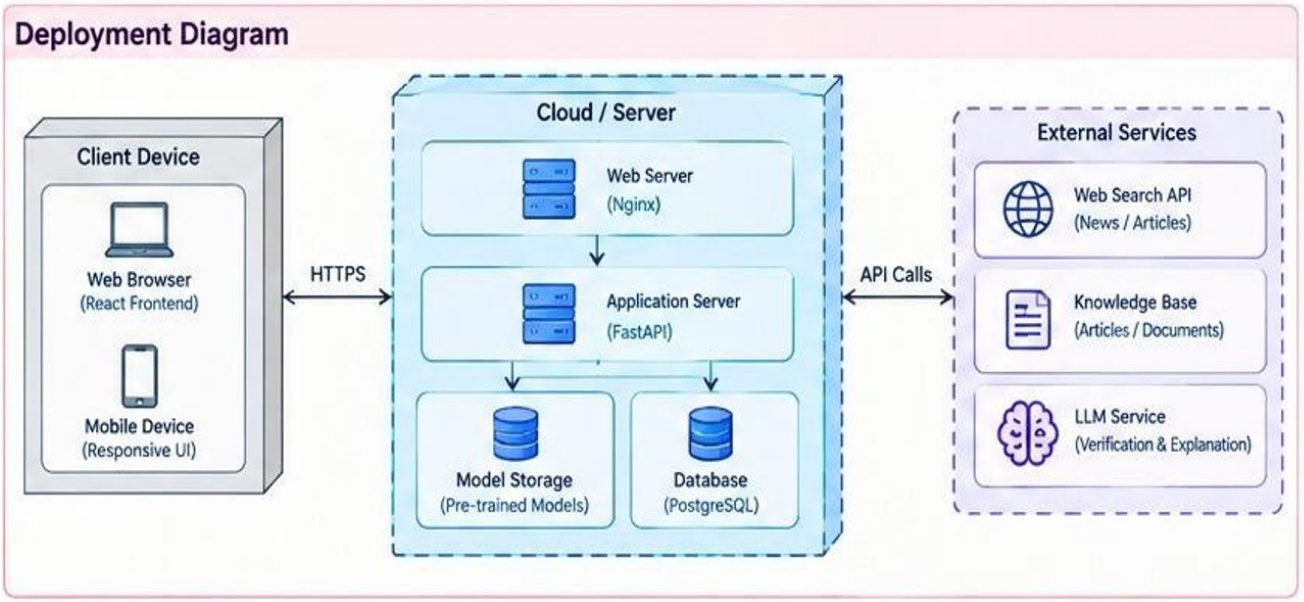


Fig 4.4 Deployment diagram

4.3. Technologies Used

Layer	Technology	Purpose in MedRAG+
Image features	Vision model,VIT	Feature extraction from the optional news image
Embeddings	BERT	Semantic representation of the claim and for retrieving relevant articles
Entity extraction	spaCy	Finding the entities used as graph nodes
Graph learning	Graph Attention Network (GAT)	Learning the importance of relationships in the graph and predicting credibility
Evidence retrieval	Web Search API, Knowledge Base	Searching, fetching and ranking evidence from trusted sources
Language model	LLM service	Verifying the verdict and generating the explanation
Backend	Nginx	Receiving HTTPS requests and passing them to the application server
Database	PostgreSQL	Users, sessions, messages and safety flags
Frontend	React 19, Vite, Tailwind CSS 4	Claim entry form, result display and responsive interface for desktop and mobile
Testing	pytest	Automated tests of the API and pipeline stages

Table 4.1 Technologies usedPython and Flask.

REFERENCES

1. S. Harris, V. T. Ta, M. Trovati, G. Nakhla, F. Latif and I. Korkontzelos, "Multimodal misinformation detection across diverse languages using RAG and LLMs," *Journal of Intelligent Information Systems*, Springer, 2026, doi: 10.1007/s10844-026-01042-x.
2. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
3. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171-4186.
4. P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò and Y. Bengio, "Graph Attention Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.10903.
5. Y. Wang et al., "EANN: Event Adversarial Neural Networks for Multi-Modal Fake News Detection," in *Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 849-857, doi: 10.1145/3219819.3219903.
6. S. Singhal, R. R. Shah, T. Chakraborty, P. Kumaraguru and S. Satoh, "SpotFake: A Multi-modal Framework for Fake News Detection," in *Proc. IEEE Fifth International Conference on Multimedia Big Data (BigMM)*, 2019, pp. 39-47, doi: 10.1109/BigMM.2019.00-44.
7. J. Jing et al., "Multimodal fake news detection via progressive fusion networks," *Information Processing & Management*, vol. 60, no. 1, art. 103120, 2023, doi: 10.1016/j.ipm.2022.103120.
8. S. Nie and Z. Zeng, "Towards real-world multimodal propagation networks: Multimodal uncertainty graph contrastive learning for fake news detection," *Journal of Intelligent Information Systems*, 2025, doi: 10.1007/s10844-025-00987-9.
9. B. M. Yao et al., "End-to-End Multimodal Fact-Checking and Explanation Generation: A Challenging Dataset and Models," in *Proc. 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023, pp. 2733-2743, doi: 10.1145/3539618.3591879.
10. S. Tahmasebi, E. Müller-Budack and R. Ewerth, "Multimodal Misinformation Detection Using Large Vision-Language Models," in *Proc. 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, pp. 2189-2199, doi: 10.1145/3627673.3679826.
11. K. Huang, X. Li and S. Uddin, "Enhancing fake news detection through fact-augmented LLM generation with co-attention," *Journal of Intelligent Information Systems*, vol. 64, no. 1, pp. 425-443, 2026, doi: 10.1007/s10844-025-01007-6.
12. J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.